

Experiment7 Report

R. VIKAS RAJ, EE19B108, Batch 1

October 2024

Objectives

- Using embedded C to program an ARM-based microcontroller.
- I/O programming in ARM microcontroller using C.
- Stepper motor control, DIP switch input reading and LED control in LPC2148.

Software used

- Keil microvision 5
- flash magic

Hardware used

- ARK's LPC2148 embedded ARM development board
- RS-232 cable
- Stepper motor

Displaying the last 3 digits of Roll-number on the LEDs Code

```
1 // LPC2148 - C program to display a value on the LEDs
2 #include "LPC214x.h" /* LPC21xx definitions */
3
4 #define LED_IOPIN IO0PIN
5 #define BIT ( x ) ( 1 << x )
6
7 #define LED_D0 ( 1 << 10 ) // P0 .10
8 #define LED_D1 ( 1 << 11 ) // P0 .11
9 #define LED_D2 ( 1 << 12 ) // P0 .12
10 #define LED_D3 ( 1 << 13 ) // P0 .13
11
12 #define LED_D4 ( 1 << 15 ) // P0 .15
13 #define LED_D5 ( 1 << 16 ) // P0 .16
14 #define LED_D6 ( 1 << 17 ) // P0 .17
15 #define LED_D7 ( 1 << 18 ) // P0 .18
16 #define LED_DATA_MASK (( unsigned long ) ( LED_D7 | LED_D6 | LED_D5 | LED_D4 | LED_D3 |
    LED_D2 | LED_D1 | LED_D0 ))
17 #ifndef LED_DRIVER_OUTPUT_EN
18 #define LED_DRIVER_OUTPUT_EN ( 1 << 5 ) // P0 .5
19 #endif
20 #define LED1_ON LED_IOPIN |= ( unsigned long ) ( LED_D0 ) ; // LED1 ON
21 #define LED2_ON LED_IOPIN |= ( unsigned long ) ( LED_D1 ) ; // LED2 ON
22 #define LED3_ON LED_IOPIN |= ( unsigned long ) ( LED_D2 ) ; // LED3 ON
23 #define LED4_ON LED_IOPIN |= ( unsigned long ) ( LED_D3 ) ; // LED4 ON
24 #define LED5_ON LED_IOPIN |= ( unsigned long ) ( LED_D4 ) ; // LED5 ON
25
26 #define LED6_ON LED_IOPIN |= ( unsigned long ) ( LED_D5 ) ; // LED6 ON
27 #define LED7_ON LED_IOPIN |= ( unsigned long ) ( LED_D6 ) ; // LED7 ON
28 #define LED8_ON LED_IOPIN |= ( unsigned long ) ( LED_D7 ) ; // LED8 ON
29
30 int main ( void )
31 {
32     IO0DIR |= LED_DATA_MASK ;
```

```

33 IO0DIR |= LED_DRIVER_OUTPUT_EN ;
34 IO0CLR |= LED_DRIVER_OUTPUT_EN ;
35
36
37 while (1)
38 {
39     int value = 0x2a ;
40
41     if( value & BIT (0) ) LED8_ON ;
42     if( value & BIT (1) ) LED7_ON ;
43     if( value & BIT (2) ) LED6_ON ;
44     if( value & BIT (3) ) LED5_ON ;
45
46     if( value & BIT (4) ) LED4_ON ;
47     if( value & BIT (5) ) LED3_ON ;
48     if( value & BIT (6) ) LED2_ON ;
49     if( value & BIT (7) ) LED1_ON ;
50 }
51
52 return 0;
53 }

```

Explanation

- The code includes the necessary header file 'LPC214x.h' for accessing the register definitions of the LPC2148 microcontroller.
- Several macros are defined to represent:
 - 'BIT(x)': A macro that shifts '1' by 'x' positions to easily create bit masks.
 - 'LED_D0' to 'LED_D7': Constants that define each LED bit position on Port 0 pins (P0.10 to P0.18).
 - 'LED_DATA_MASK': A mask combining all LED bits to set up as outputs.
 - 'LED_DRIVER_OUTPUT_EN': Represents P0.5, which controls enabling output to LEDs.
 - 'LED1_ON' to 'LED8_ON': Macros to turn each LED on by setting the corresponding bit in 'IO0PIN'.
- The 'main' function begins by configuring the LED pins as output:
 - 'IO0DIR |= LED_DATA_MASK' sets the pins defined by 'LED_DATA_MASK' as outputs. –
 - 'IO0DIR |= LED_DRIVER_OUTPUT_EN' enables output on P0.5 for LED driver control. – 'IO0CLR |= LED_DRIVER_OUTPUT_EN' clears P0.5 to activate the LED driver.
- Inside the infinite 'while(1)' loop:
 - An integer variable 'value' is set to '0x2A' (binary '00101010'), which is my roll number. –
 - Bitwise operations are used to check each bit in 'value':
 - * For each bit, if 'value & BIT(n)' is true, the corresponding LED ('LED8' to 'LED1') is turned on using the macros 'LEDx ON'.
- The program runs indefinitely, keeping specific LEDs on based on the 'value' variable, which in this case is 42.
- NOTE: There is no LEDx OFF defined since it is of no use here.

Reading inputs from a DIP-Switch and displaying Code

```

1
2
3 #include "LPC214x.H" /* LPC214x definitions */ #include "led.h"
4 #include "delay.h"
5
6
7
8 #define DIP_SW_D0 (1 << 16) // P0.16
9 #define DIP_SW_D1 (1 << 17) // P0.17
10 #define DIP_SW_D2 (1 << 18) // P0.18
11 #define DIP_SW_D3 (1 << 19) // P0.19
12

```

```

13 #define DIP_SW_D4 (1 << 22) // P0 .22
14 #define DIP_SW_D5 (1 << 23) // P0 .23
15 #define DIP_SW_D6 (1 << 24) // P0 .24
16 #define DIP_SW_D7 (1 << 25) // P0 .25
17
18 #define DIP_SW_DIR IO1DIR
19 #define DIP_SW_PIN IO1PIN
20
21 #define DIP_SW_DATA_MASK ( DIP_SW_D7 | DIP_SW_D6 | DIP_SW_D5 | DIP_SW_D4 | DIP_SW_D3 |
    DIP_SW_D2 | DIP_SW_D1 | DIP_SW_D0 )
22
23 void set_dipswitch_port_input ( void )
24 {
25     DIP_SW_DIR &= ~( DIP_SW_DATA_MASK ) ;
26 }
27
28 unsigned long read_dip_switch ( void )
29 {
30     return DIP_SW_PIN ;
31 }
32
33 int main ( void )
34 {
35     unsigned long sw_status ;
36
37     set_led_port_output () ;
38     set_dipswitch_port_input () ;
39
40     while (1)
41     {
42         sw_status = read_dip_switch () ;
43
44         if( sw_status & DIP_SW_D0 ) { LED1_OFF ; } else { LED1_ON ; } 45 delay_mSec (10) ;
46         if( sw_status & DIP_SW_D1 ) { LED2_OFF ; } else { LED2_ON ; } 47 delay_mSec (10) ;
48         if( sw_status & DIP_SW_D2 ) { LED3_OFF ; } else { LED3_ON ; } 49 delay_mSec (10) ;
49         if( sw_status & DIP_SW_D3 ) { LED4_OFF ; } else { LED4_ON ; } 51 delay_mSec (10) ;
52
53         if( sw_status & DIP_SW_D4 ) { LED5_OFF ; } else { LED5_ON ; } 54 delay_mSec (10) ;
55         if( sw_status & DIP_SW_D5 ) { LED6_OFF ; } else { LED6_ON ; } 56 delay_mSec (10) ;
57         if( sw_status & DIP_SW_D6 ) { LED7_OFF ; } else { LED7_ON ; }
58         delay_mSec (10) ;
59         if( sw_status & DIP_SW_D7 ) { LED8_OFF ; } else { LED8_ON ; } 60 delay_mSec (10) ;
61     }
62
63     return 0;
64 }

```



Explanation

- The program includes necessary header files for LPC2148 definitions, as well as custom header files for LED ('led.h') and delay functions ('delay.h').
- Macros are defined for DIP switch pins:
 - 'DIP SW D0' to 'DIP SW D7': These macros define DIP switch bits on Port 0 from pins P0.16 to P0.25.
 - 'DIP SW DIR' and 'DIP SW PIN': Aliases for setting and reading Port 1 direction and pin values.
 - 'DIP SW DATA MASK': A mask combining all DIP switch bits to configure them as inputs.
- The function 'set dipswitch port input' sets the direction of the DIP switch pins: – 'DIP SW DIR &= ~(DIP SW DATA MASK);' clears the DIP switch bits in 'DIP SW DIR', configuring them as inputs.
- The function 'read dip switch' reads the status of all DIP switches:
 - 'return DIP SW PIN;' reads and returns the current status of Port 1.
- In the 'main' function:
 - 'set led port output()' and 'set dipswitch port input()' configure the LED and DIP switch ports as output and input, respectively.
 - 'sw status' is a variable to store the DIP switch status in each iteration.
- Inside the 'while(1)' loop:
 - 'sw status = read dip switch();' updates the DIP switch status.
 - Each DIP switch state (D0 to D7) is checked using 'if' statements:
 - * If a switch bit is set, the corresponding LED is turned off.
 - * If a switch bit is cleared, the corresponding LED is turned on.
 - * After each 'if' statement, 'delay mSec(10);' introduces a 10ms delay to avoid rapid toggling.

Adding two nibbles by reading from a DIP switch and displaying on the LEDs

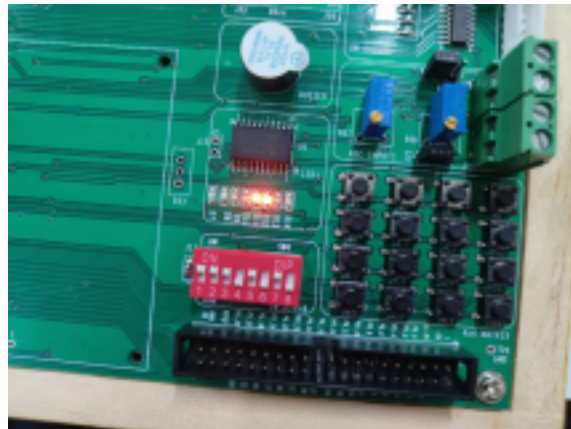
Code

```
1
2 #include "LPC214x.H" /* LPC214x definitions */ #include "led.h"
3
4 #include "delay.h"
5
6
7 #define DIP_SW_D0 (1 << 16) // P0.16
8 #define DIP_SW_D1 (1 << 17) // P0.17
9 #define DIP_SW_D2 (1 << 18) // P0.18
10 #define DIP_SW_D3 (1 << 19) // P0.19
11
12 #define DIP_SW_D4 (1 << 22) // P0.22
13 #define DIP_SW_D5 (1 << 23) // P0.23
14 #define DIP_SW_D6 (1 << 24) // P0.24
15 #define DIP_SW_D7 (1 << 25) // P0.25
16
17 #define DIP_SW_DIR IO1DIR
18 #define DIP_SW_PIN IO1PIN
19
20 #define DIP_SW_DATA_MASK ( DIP_SW_D7 | DIP_SW_D6 | DIP_SW_D5 | DIP_SW_D4 | DIP_SW_D3 |
    DIP_SW_D2 | DIP_SW_D1 | DIP_SW_D0 )
21
22
23
24 void set_dipswitch_port_input ( void )
25 {
26     DIP_SW_DIR &= ~( DIP_SW_DATA_MASK );
27 }
28
```

```

29 unsigned long read_dip_switch ( void )
30 {
31 return DIP_SW_PIN ;
32 }
33
34
35 int main ( void )
36 {
37 unsigned long sw_status ;
38
39 set_led_port_output () ;
40 set_dipswitch_port_input () ;
41
42 while (1)
43 {
44 sw_status = ~ read_dip_switch () ;
45 unsigned long num1 = sw_status ;
46 unsigned long num2 = ( sw_status >> 6) & 0 x000F0000 ; 47 num1 &=0 x000F0000 ;
48 unsigned long sum = num1 + num2 ;
49 if( sum & DIP_SW_D0 ) { LED8_ON ;} else { LED8_OFF ;} 50 delay_mSec (10) ;
51 if( sum & DIP_SW_D1 ) { LED7_ON ;} else { LED7_OFF ;} 52 delay_mSec (10) ;
53 if( sum & DIP_SW_D2 ) { LED6_ON ;} else { LED6_OFF ;} 54 delay_mSec (10) ;
55 if( sum & DIP_SW_D3 ) { LED5_ON ;} else { LED5_OFF ;} 56 delay_mSec (10) ;
57
58 if( sum & ( DIP_SW_D4 > >2) ) { LED4_ON ;} else { LED4_OFF ;} 59 delay_mSec (10) ;
60 if( sum & ( DIP_SW_D5 > >2) ) { LED3_ON ;} else { LED3_OFF ;} 61 delay_mSec (10) ;
62 if( sum & ( DIP_SW_D6 > >2) ) { LED2_ON ;} else { LED2_OFF ;} 63 delay_mSec (10) ;
64 if( sum & ( DIP_SW_D7 > >2) ) { LED1_ON ;} else { LED1_OFF ;} 65 delay_mSec (10) ;
66
67 delay_mSec (1) ;
68 }
69
70 return 0;
71 }

```



Explanation

- The program includes necessary header files for the LPC2148 definitions and custom header files for LED ('led.h') and delay functions ('delay.h').
- Macros are defined for DIP switch pins:
 - 'DIP SW D0' to 'DIP SW D7' are defined as bit-shifted values corresponding to pins P0.16 to P0.25 on Port 0.
 - 'DIP SW DIR' and 'DIP SW PIN' are used as aliases for setting and reading Port 1 direction and pin values.
 - 'DIP SW DATA MASK' is a bitwise OR of all DIP switch macros, which is used to configure all these bits together as inputs.
- The function 'set dipswitch port input' configures the DIP switch pins as input by clearing the bits specified in 'DIP SW DATA MASK' in 'DIP SW DIR'.
- The function 'read dip switch' reads the status of all DIP switches by returning the value of 'DIP SW PIN'.

In the 'main' function:

- ‘set_led_port_output()’ and ‘set_dipswitch_port_input()’ are called to set the LED port as output and the DIP switch port as input, respectively.
- ‘sw_status’ is declared as a variable to store the DIP switch status. Since the DIP switches on the board are configured so that ON means a low signal, a bitwise NOT (‘ ~ ’) is applied to ‘sw_status’ to invert the reading.
- The program processes the switch values as two 4-bit numbers:
 - ‘num1’ is the lower 4 bits of ‘sw_status’, masked by ‘0x000F0000’.
 - ‘num2’ is obtained by shifting ‘sw_status’ by 6 bits to access another 4-bit value, then masking it with ‘0x000F0000’.
 - ‘sum’ is calculated by adding ‘num1’ and ‘num2’.
- In the ‘while(1)’ loop:
 - For each LED, the code checks specific bits in ‘sum’:
 - * For example, ‘if (sum & DIP_SW_D0)’ turns on ‘LED8’ if the least significant bit of ‘sum’ is set.
 - * LEDs ‘LED8’ to ‘LED1’ are turned on/off based on bits in ‘sum’, using delays of 10ms after each check.
 - * Bits in ‘DIP_SW_D4’ to ‘DIP_SW_D7’ are checked after shifting them by two positions for the higher LEDs.
 - ‘delay_mSec(1);’ introduces a short delay after each loop iteration to control the LED update speed.

[H]

Multiplying two nibbles by reading from a DIP switch and displaying on the LEDs

Code

```

1
2 #include "LPC214x.H" /* LPC214x definitions */ #include "led.h"
3
4 #include "delay.h"
5
6
7
8 #define DIP_SW_D0 (1 << 16) // P0 .16
9 #define DIP_SW_D1 (1 << 17) // P0 .17
10 #define DIP_SW_D2 (1 << 18) // P0 .18
11 #define DIP_SW_D3 (1 << 19) // P0 .19
12
13 #define DIP_SW_D4 (1 << 22) // P0 .22
14 #define DIP_SW_D5 (1 << 23) // P0 .23
15 #define DIP_SW_D6 (1 << 24) // P0 .24
16 #define DIP_SW_D7 (1 << 25) // P0 .25
17
18 #define DIP_SW_DIR IO1DIR
19 #define DIP_SW_PIN IO1PIN
20
21 #define DIP_SW_DATA_MASK ( DIP_SW_D7 | DIP_SW_D6 | DIP_SW_D5 | DIP_SW_D4 | DIP_SW_D3 |
    DIP_SW_D2 | DIP_SW_D1 | DIP_SW_D0 )
22
23
24
25 void set_dipswitch_port_input ( void )
26 {
27     DIP_SW_DIR &= ~( DIP_SW_DATA_MASK ) ;
28 }
29
30 unsigned long read_dip_switch ( void )
31 {
32     return DIP_SW_PIN ;
33 }
34
35 int main ( void )

```

```

36 {
37 unsigned long sw_status ;
38
39 set_led_port_output () ;
40 set_dipswitch_port_input () ;
41
42 while (1)
43 {
44 sw_status = ~ read_dip_switch () ;
45 unsigned long num1 = sw_status ;
46 unsigned long num2 = ( sw_status >> 6) & 0 x000F0000 ; 47 num1 &=0 x000F0000 ;
48 num1 = num1 >>16;
49 num2 = num2 >>16;
50 unsigned long product = num1 * num2 ;
51 product = product <<16;
52 if( product & DIP_SW_D0 ) { LED8_ON ;} else { LED8_OFF ;} 53 delay_mSec (10) ;
54 if( product & DIP_SW_D1 ) { LED7_ON ;} else { LED7_OFF ;} 55 delay_mSec (10) ;
56 if( product & DIP_SW_D2 ) { LED6_ON ;} else { LED6_OFF ;} 57 delay_mSec (10) ;
58 if( product & DIP_SW_D3 ) { LED5_ON ;} else { LED5_OFF ;} 59 delay_mSec (10) ;
60
61 if( product & ( DIP_SW_D4 >>2) ) { LED4_ON ;} else { LED4_OFF ;} 62 delay_mSec (10) ;
63 if( product & ( DIP_SW_D5 >>2) ) { LED3_ON ;} else { LED3_OFF ;} 64 delay_mSec (10) ;
65 if( product & ( DIP_SW_D6 >>2) ) { LED2_ON ;} else { LED2_OFF ;} 66 delay_mSec (10) ;
67 if( product & ( DIP_SW_D7 >>2) ) { LED1_ON ;} else { LED1_OFF ;} 68 delay_mSec (10) ;
69
70 delay_mSec (1) ;
71 }
72
73 return 0;
74 }

```

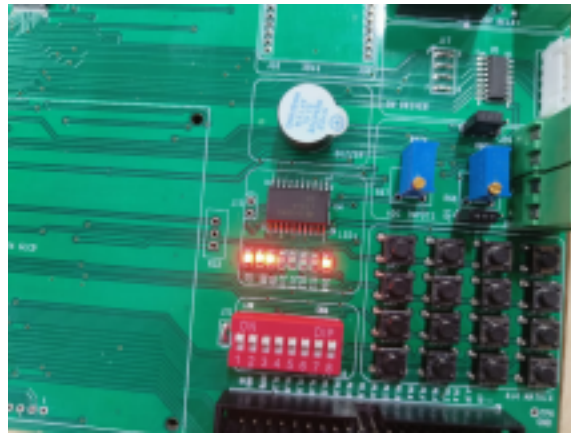
Explanation

- The program includes header files for LPC2148 definitions, LED control ('led.h'), and delay functions ('delay.h').
- Macros are defined for DIP switch pins:
 - 'DIP SW D0' to 'DIP SW D7' are defined as bit-shifted values, corresponding to pins P0.16 to P0.25 on Port 0.
 - 'DIP SW DIR' and 'DIP SW PIN' are aliases for setting and reading Port 1 direction and pin values, respectively.
 - 'DIP SW DATA MASK' is a bitwise OR of all DIP switch macros, allowing all pins to be configured as inputs.
- The function 'set dipswitch port inp_ut' configures the DIP switch pins as input by clearing the bits in 'DIP SW DIR' corresponding to 'DIP SW DATA MASK'.
- The function 'read dip switch' returns the current state of the DIP switches by reading 'DIP SW PIN'.

In the 'main' function:

- 'set led port output()' and 'set dipswitch port inp_ut()' are called to set the LED port as output and the DIP switch port as input, respectively.
- 'sw status' is declared to store the DIP switch state. A bitwise NOT ('~ ') is applied to invert 'sw status' because the DIP switches are configured so that the ON position outputs a low signal.
 - * 'sw status = ~read dip switch();'
- 'num1' is assigned the 4-bit value from DIP switches D0 to D3, masked by '0x000F0000'.
- 'num2' is assigned the 4-bit value from DIP switches D4 to D7, shifted right by 6 bits and masked by '0x000F0000'.
- Both 'num1' and 'num2' are shifted right by 16 bits to isolate the intended values before performing the multiplication.
- The product of 'num1' and 'num2' is stored in 'product', which is then shifted left by 16 bits to position it correctly for LED output.
- In the 'while(1)' loop:

- LEDs are controlled based on the bits in 'product':
- A short delay 'delay_mSec(1);' at the end of each loop iteration helps control the LED update speed.



Stepper motor-control

Code

```

1 #include < LPC214x .h > /* LPC21xx definitions */
2
3 void delay_mSec ( int ) ;
4
5 int main ( void )
6 {
7     int i ;
8     unsigned char steps [4] = { 0 x09 , 0 x0c , 0 x06 , 0 x03 }; // standard step sequence for stepper
9                             motor
10     signed char x = 0;
11
12     PINSEL0 = 0 x0 ; // Pin function Select -> P0 .0 to P0 .15 -> GPIO Port
13     IO0DIR |= 0 xF0 ; // Set stepper motor pins as output in IO0 port
14     delay_mSec (10) ;
15
16     while (1)
17     {
18         for( i =0; i <2500; i ++ )
19         {
20             IO0PIN = ( steps [ x ++] << 4) ; // send the 4 bit step value to stepper motor lines connected to IO0 port
21             if( x > 3)
22                 x = 0;
23
24             delay_mSec (2) ;
25         }
26     }
27     return 0;
28 }
29
30 void delay_mSec ( int dCnt ) // pr_note :~ dCnt mSec
31 {
32     int j =0 , i =0;
33     while ( dCnt -- )
34     {
35         for ( j =0; j <1000; j ++ )
36         {
37             /* At 60 Mhz , the below loop introduces
38             delay of 10 us */
39             for( i =0; i <10; i ++ ) ;
40         }
41     }
42 }

```


Explanation

- This program controls a stepper motor connected to an LPC214x microcontroller using a standard 4-step sequence.
- An array 'steps[4]' is defined to store the stepper motor control sequence in 4-bit values: – The sequence 0x09, 0x0C, 0x06, 0x03 is used to control the four phases of the stepper motor.
- In 'main', the following setup is performed:
 - 'PINSEL0 = 0x0;' configures pins P0.0 to P0.15 as general-purpose I/O.
 - 'IO0DIR |= 0xF0;' sets P0.4 to P0.7 as output pins for driving the stepper motor.
 - A short delay 'delay_mSec(10);' is introduced to stabilize the setup.
- The 'while(1)' loop contains a nested 'for' loop that executes 2500 times to complete a series of motor steps:
 - The current step is sent to the motor, after shifting the 4-bit step sequence to align with pins P0.4 to P0.7.
 - 'x' increments after each step, and when x is greater than 3, it resets to 0, looping through the step sequence repeatedly.
 - 'delay_mSec(2);' creates a small delay to control the step rate.
- The 'delay_mSec' function generates a delay in milliseconds:
 - The outer 'while(dCnt--)' loop controls the duration in milliseconds.
 - The inner loops introduce a delay of approximately 10 microseconds per iteration.



Conclusion

Using C programming to program a microcontroller makes it easier for a programmer to write the program since one doesn't have to deal with only hex or binary values and can directly use decimal values while programming. Embedded-C also operates very close to the hardware making it more reliable (although not more than assembly).

The usage of thumb instructions by the processor makes it more memory efficient. The 48 PINS on the ARM processor can be configured to do up to 4 tasks including GPIO. This allows for a wide range of processor applications. The configuration of GPIO pins in ARM is very similar to that of AVR.

The presence of an integrated driver for the Stepper motor in LPC2148 and the pre-made connections between the processor and various I/O devices like the DIP-Switch, LED and the button arrays makes it easier to prototype.